

9세션 챌린지: Architecture Keywords to Compose

이 챌린지는 Day 5에서 실행한 Compose architecture template을 바탕으로, 주어진 아키텍처 키워드만 보고 직접 `compose.yaml`을 설계하는 자율 실습이다. 목표는 정답 파일을 베끼는 것이 아니라, service boundary, network area, traffic path, stateful service, 부하 성향을 스스로 판단해 실행 가능한 Compose 구조로 만드는 것이다.

Challenge Goal

- 주어진 architecture keyword set 중 하나를 선택한다.
- 최소 4개 이상의 service를 가진 Compose stack을 직접 만든다.
- 외부 진입점, 내부 service name, network area, volume, runtime config를 설명한다.
- `curl`, `docker compose ps`, `logs`, `DB/Redis/queue` 확인 결과를 기록한다.
- traffic/CPU/memory pressure가 어느 service에 몰릴지 운영 관점으로 비교한다.

작업 위치

```
cd week2/day5/labs/compose-architecture-challenge
```

권장 파일:

파일	내용
<code>compose.yaml</code>	직접 작성한 Compose stack
<code>README.md</code>	실행 방법과 구조 설명
<code>NOTES.md</code>	증거와 판단을 표로 정리
<code>apps/</code> 또는 <code>html/</code>	필요한 최소 앱 코드

진행 원칙

이번 챌린지는 예시 `compose`를 제공하지 않는다. 대신 keyword, 제약, 증거 기준만 제공한다.

허용:

- `nginx`, `node`, `postgres`, `redis`, `postgrest`, `adminer`, `busybox` 같은 공개 image 사용
- 아주 작은 Node.js API 또는 static HTML 작성
- Dockerfile 작성 없이 bind mount로 app code 실행
- `profiles`로 `tool/admin/worker` 선택 실행 구성

제한:

- 모든 service를 default network 하나에 넣지 않는다.
- DB/PostgreSQL은 host port로 공개하지 않는다.
- 내부 service끼리 localhost로 연결하지 않는다.
- secret 값 또는 password를 기록 문서에 그대로 노출하지 않는다.
- docker compose config만 보고 끝내지 않고 실제 실행 증거를 남긴다.

Architecture Keyword Sets

강사는 팀별로 하나를 지정하거나 학생이 하나를 선택하게 한다.

Set A: Commerce Catalog Plus

키워드:

```
frontend, catalog-api, postgres, redis-cache, cache-writer, public_net,
app_net, cache_net, data_net
```

요구 흐름:

```
browser -> frontend
frontend/api check -> catalog-api
catalog-api -> postgres
cache-writer -> redis-cache
```

부하 관찰 힌트:

관점	예상
traffic 집중	frontend, catalog-api
CPU 후보	catalog-api의 JSON/query 처리
memory/state 후보	postgres buffer/cache, redis key memory
먼저 볼 증거	/products, Redis key, DB query

Set B: Order Async Worker

키워드:

```
gateway, order-api, redis-queue, worker, postgres, admin-ui(optional),
public_net, queue_net, data_net
```

요구 흐름:

```

browser/curl -> gateway -> order-api
order-api -> redis-queue
worker -> redis-queue
worker -> postgres

```

부하 관찰 힌트:

관점	예상
traffic 집중	gateway, order-api
CPU 후보	worker job 처리
memory/state 후보	redis queue backlog, postgres write
먼저 볼 증거	publish response, queue length, worker log

Set C: Internal Platform Gateway

키워드:

```

gateway, web-console, identity-api, config-api, postgres, redis-cache,
public_net, app_net, cache_net, data_net

```

요구 흐름:

```

browser -> gateway
gateway -> web-console
gateway -> identity-api
gateway -> config-api
identity-api -> postgres
config-api -> redis-cache

```

부하 관찰 힌트:

관점	예상
traffic 집중	gateway
CPU 후보	identity-api auth/permission logic
memory/state 후보	postgres user/session data, redis config/cache
먼저 볼 증거	route별 curl, gateway log, Redis/DB check

Set D: Event Processing Mini Platform

키워드:

```
ingest-api, redis-stream-or-list, processor-worker, result-api, postgres,
public_net, queue_net, data_net
```

요구 흐름:

```
curl -> ingest-api -> redis queue
processor-worker -> redis queue
processor-worker -> postgres
curl -> result-api -> postgres
```

부하 관찰 힌트:

관점	예상
traffic 집중	ingest-api, result-api
CPU 후보	processor-worker
memory/state 후보	queue backlog, postgres result table
먼저 볼 증거	enqueue result, worker log, result query

필수 설계 조건

항목	최소 조건
Service 수	4개 이상
Network	public_net과 내부 network 1개 이상 분리
External endpoint	host port는 gateway/frontend/API 중 필요한 endpoint만 공개
Stateful service	DB 또는 Redis 중 1개 이상
Volume	PostgreSQL 사용 시 named volume 사용
Runtime config	service name 기반 env 사용
Evidence	HTTP, logs, DB/Redis/queue 중 3종 이상
Cleanup	down과 down -v 차이 설명

설계 순서

1. Keyword set에서 service 후보를 모두 적는다. 2. 외부 사용자가 들어오는 service를 하나로 정한다.
3. stateful service를 고르고 host에 공개하지 않는다. 4. network area를 public, app, cache/queue, data로 나눈다. 5. service 간 주소는 Compose service name으로만 쓴다. 6. 최소 동작 앱을 만든다.
7. docker compose config로 구조를 검증한다. 8. docker compose up -d로 실행한다. 9. HTTP/log/DB/Redis/queue 증거를 남긴다. 10. traffic/CPU/memory pressure를 표로 비교한다.

실행 증거 명령 예시

명령은 그대로 베끼는 것이 아니라 본인 구조에 맞게 바꾼다.

```
docker compose config
docker compose up -d
docker compose ps
curl -s http://localhost:<entry-port>/<path>
docker compose logs <service> --tail 40
docker compose exec <db-service> psql -U <user> -d <db> -c "<query>"
docker compose exec <redis-service> redis-cli LLEN <queue>
docker compose exec <redis-service> redis-cli GET <key>
```

실패 주입

최소 1개 이상 수행한다.

실패	기대 관찰
backend/API 중지	gateway/frontend는 살아 있지만 특정 route 실패
DB 중지	API는 running이어도 정상 응답 실패
Redis 중지	cache/queue 기능 실패
wrong service name env	container는 떠도 dependency 연결 실패
port 충돌	compose up 단계에서 host port 오류

기록 표

labs/compose-architecture-challenge/NOTES.md에 정리한다.

항목	작성 내용
선택한 keyword set	A/B/C/D 중 하나
내가 만든 service 목록	service name과 역할
외부 진입점	host port와 path
내부 연결	service name 기반 연결
network area	public/app/cache/queue/data 구분
stateful service	DB/Redis/volume
traffic 집중 지점	이유 포함
CPU 부하 후보	이유 포함
memory/state 부하 후보	이유 포함
정상 실행 증거	curl/log/DB/Redis/queue 결과
실패 주입 결과	증상, 첫 확인 명령, 복구
cleanup 판단	down 또는 down -v 선택 이유

핵심 포인트

이 챌린지의 정답은 하나가 아니다. 중요한 것은 architecture keyword를 보고 service boundary를 정하고, Compose network와 volume으로 구조를 표현하며, 실행 증거와 부하 성향까지 설명하는 것이다.